



Karnataka ReddyJana Sangha®
VEMANA INSTITUTE OF TECHNOLOGY

Approved by AICTE-New Delhi, Affiliated to VTU-Belagavi, Recognized by Govt. of Karnataka
#1, Mahayogi Vemana Road, 3rd Block, Koramangala, Bengaluru - 560034

www.veமானait.edu.in



DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Mini Project Report

on

STUDENT AND COURSE DATABASE

Submitted as a part of the requirements for the course

Database Management Systems

offered by

Department of Artificial Intelligence and Machine Learning

Submitted By

SPOORTHIM	1VI24AI043
SRINIDHI U NAIK	1VI24AI044
SRIRAKSHA NR	1VI24AI045
SUKUMAR P	1VI24AI046
SURYA PRAKASH V	1VI24AI047

Guided By

Lakshmi Sravani G

Assistant Professor

VEMANA INSTITUTE OF TECHNOLOGY

(Affiliated to VTU & Approved by AICTE)

#1, Mahayogi Vemana Road, Koramangala, Bengaluru – 560034

Academic Year 2025–2026

PROBLEM STATEMENT

Consider the schema for Student and Course Database:

- **STUDENT** (Name, SNumber, Class, Major)
- **COURSE** (CName, CNumber, Credit_Hrs, Dept)
- **SECTION** (SectionID, CNumber, Sem, Year, Instructor)
- **GRADE_REPORT** (SNumber, SectionID, Grade)
- **PREREQUISITE** (CNumber, Prerequisite_Number)

Solve the following queries using SQL:

1. Retrieve the names of all senior students majoring in 'cs'.
2. Retrieve the names of all courses taught by Professor King in 2016 and 2017.
3. For each section taught by Professor King, retrieve the course number, semester, year, and number of students who took the section.
4. Display student names and their grades.
5. Find courses with credit hours greater than 3.

INTRODUCTION

A Student and Course Database Management System is used to store and manage information related to students, courses, sections, grades, and prerequisites within an educational institution.

Managing academic information manually can be difficult, time-consuming, and prone to errors. A relational database provides a structured approach for storing, retrieving, and maintaining data while ensuring accuracy, consistency, and reduced redundancy.

The database consists of five related tables: STUDENT, COURSE, SECTION, GRADE_REPORT, and PREREQUISITE. These tables are connected through primary key and foreign key relationships.

Together, they store student details, course information, section schedules, grade records, and prerequisite requirements, forming a complete academic database system.

This mini project demonstrates the use of SQL for retrieving and analyzing academic information. The project highlights the practical application of relational database concepts and SQL in academic data management.

Table Creation

STUDENT

```
CREATE TABLE STUDENT (  
    name VARCHAR(50),  
    snumber INT PRIMARY KEY,  
    class VARCHAR(20),  
    major VARCHAR(20)  
);
```

COURSE

```
CREATE TABLE COURSE (  
    cname VARCHAR(50),  
    cnumber INT PRIMARY KEY,  
    credit_hrs INT,  
    dept VARCHAR(20)  
);
```

SECTION

```
CREATE TABLE SECTION (  
    sectionid INT PRIMARY KEY,  
    cnumber INT,  
    sem VARCHAR(20),  
    year INT,  
    instructor VARCHAR(50),  
    FOREIGN KEY (cnumber)  
    REFERENCES COURSE(cnumber)  
);
```

GRADE_REPORT

```
CREATE TABLE GRADE_REPORT (  
    snumber INT,  
    sectionid INT,  
    grade CHAR(2),  
    FOREIGN KEY (snumber)  
    REFERENCES STUDENT(snumber),  
    FOREIGN KEY (sectionid)  
    REFERENCES SECTION(sectionid)  
);
```

PREREQUISITE

```
CREATE TABLE PREREQUISITE (  
    cnumber INT,  
    prerequisite_number INT,  
    FOREIGN KEY (cnumber)  
    REFERENCES COURSE(cnumber)  
);
```

Data Insertion

STUDENT

```
INSERT INTO STUDENT VALUES ('Rahul', 1, 'senior', 'cs');  
INSERT INTO STUDENT VALUES ('Anjali', 2, 'junior', 'ece');  
INSERT INTO STUDENT VALUES ('Vikas', 3, 'senior', 'cs');
```

COURSE

```
INSERT INTO COURSE VALUES ('DBMS', 101, 4, 'CSE');  
INSERT INTO COURSE VALUES ('OS', 102, 3, 'CSE');
```

SECTION

```
INSERT INTO SECTION VALUES (1, 101, 'Odd', 2016, 'king');  
INSERT INTO SECTION VALUES (2, 102, 'Even', 2017, 'king');
```

GRADE_REPORT

```
INSERT INTO GRADE_REPORT VALUES (1, 1, 'A');  
INSERT INTO GRADE_REPORT VALUES (3, 1, 'B');  
INSERT INTO GRADE_REPORT VALUES (2, 2, 'A');
```

PREREQUISITE

```
INSERT INTO PREREQUISITE VALUES (102, 101);
```

Commit Transaction

```
COMMIT;
```

Question 1

Retrieve the names of all senior students majoring in 'cs'.

Explanation

The STUDENT table contains information about students, including their name, student number, class, and major. In many academic databases, it is often necessary to identify students belonging to a particular class and academic discipline for reporting and analysis purposes.

This query retrieves the names of students who satisfy two specific conditions. The first condition requires that the student belongs to the **senior** class, which generally represents students in the final stage of their academic program. The second condition requires that the student's major is **cs** (Computer Science).

The **SELECT** statement is used to retrieve the required attribute, namely the student's name. The **FROM** clause specifies that the data is to be obtained from the STUDENT table. The **WHERE** clause applies filtering conditions to the records. The logical operator **AND** ensures that only those records satisfying both conditions simultaneously are selected.

When the query is executed, the database scans the STUDENT table, evaluates each row against the specified conditions, and returns the names of all matching students. This type of query is commonly used for generating departmental reports, identifying eligible students for projects, and preparing academic statistics.

SQL Query

```
SELECT name
FROM STUDENT
WHERE class = 'senior'
AND major = 'cs';
```

Output

NAME
Rahul
Vikas

Result

The query was executed successfully and returned the names of students who belong to the **senior** class and are majoring in **Computer Science**. Based on the data available in the STUDENT table, the students **Rahul** and **Vikas** satisfy both conditions. Therefore, their names are displayed in the result. The output confirms that the filtering conditions specified in the WHERE clause have been applied correctly and that only the required records have been retrieved from the database.

Question 2

Retrieve the names of all courses taught by Professor King in 2016 and 2017.

Explanation

The information about courses is stored in the **COURSE** table, while details regarding course offerings, instructors, semesters, and academic years are maintained in the **SECTION** table. To determine the courses taught by Professor King during the years 2016 and 2017, it is necessary to combine information from both tables.

The query uses the **JOIN** operation to connect the **COURSE** and **SECTION** tables through the common attribute **cnumber**. This allows course names to be matched with their corresponding section details. The **WHERE** clause filters the records to include only those sections taught by the instructor **'king'** and offered during the years **2016** or **2017**.

The **IN** operator is used to specify multiple years within a single condition. The **DISTINCT** keyword ensures that duplicate course names are removed from the result, so each course appears only once even if it was taught in multiple sections. After applying these conditions, the query returns the names of all courses taught by Professor King during the specified academic years.

SQL Query

```
SELECT DISTINCT C.cname
FROM COURSE C
JOIN SECTION S
ON C.cnumber = S.cnumber
WHERE S.instructor = 'king'
AND S.year IN (2016, 2017);
```

Output

CNAME
DBMS
OS

Result

The query was executed successfully and retrieved the names of all courses taught by Professor King during the academic years 2016 and 2017. By joining the **COURSE** and **SECTION** tables and applying the specified filtering conditions, the courses **DBMS** and **OS** were identified. The use of the **DISTINCT** keyword ensured that each course name appeared only once in the final result.

Question 3

For each section taught by Professor King, retrieve the course number, semester, year, and number of students who took the section.

Explanation

The SECTION table contains details about course sections, including the course number, semester, year, and instructor. The GRADE.REPORT table stores information about students enrolled in different sections. To determine how many students attended each section taught by Professor King, data from both tables must be combined.

The query uses a **LEFT JOIN** operation between the SECTION and GRADE.REPORT tables based on the common attribute **sectionid**. A LEFT JOIN is used to ensure that all sections taught by Professor King are included in the result, even if no students are enrolled in a particular section.

The **WHERE** clause filters the records to include only those sections taught by the instructor 'king'. The **COUNT()** aggregate function is then used to count the number of student numbers associated with each section. To calculate the count separately for every section, the **GROUP BY** clause groups records according to section identifier, course number, semester, and year.

As a result, the query provides a summary showing the course number, semester, academic year, and the total number of students enrolled in each section conducted by Professor King.

SQL Query

```
SELECT S.cnumber,
       S.sem,
       S.year,
       COUNT(G.snumber) AS number_of_students
FROM SECTION S
LEFT JOIN GRADE_REPORT G
ON S.sectionid = G.sectionid
WHERE S.instructor = 'king'
GROUP BY S.sectionid,
         S.cnumber,
         S.sem,
         S.year;
```

Output

CNUMBER	SEM	YEAR	NUMBER_OF_STUDENTS
101	Odd	2016	2
102	Even	2017	1

Result

The query was executed successfully and retrieved the number of students enrolled in each section taught by Professor King.

Question 4

Display student names and their grades.

Explanation

The STUDENT table contains information about students, including their names and student numbers. The GRADE_REPORT table stores the grades obtained by students in various sections. To display student names along with their corresponding grades, the two tables must be connected using the common attribute **snumber**.

The query retrieves the student's name from the STUDENT table and the grade from the GRADE_REPORT table. The **WHERE** clause establishes the relationship between the tables by matching the student numbers. This allows the database to combine records from both tables and display the required information in a meaningful format.

SQL Query

```
SELECT ST.name,  
       G.grade  
FROM STUDENT ST, GRADE_REPORT G  
WHERE ST.snumber = G.snumber;
```

Output

NAME	GRADE
Rahul	A
Vikas	B
Anjali	A

Result

The query was executed successfully and displayed the names of students along with their respective grades.

Question 5

Find courses with credit hours greater than 3.

Explanation

The COURSE table stores information about all courses offered by the institution, including the course name, course number, credit hours, and department. Credit hours represent the academic weight of a course and are often used to determine the workload associated with a subject.

In this query, the objective is to identify courses that have more than 3 credit hours. The **SELECT** statement is used to retrieve the course name (**cname**) and the corresponding credit hours (**credit_hrs**). The **WHERE** clause applies a condition to filter only those records where the value of credit hours is greater than 3.

The database examines each record in the COURSE table and compares the credit hour value against the specified condition. Courses satisfying the condition are included in the result set, while all other courses are excluded. Such queries are useful for identifying major or high-credit courses that contribute significantly to a student's academic program.

SQL Query

```
SELECT cname, credit_hrs
FROM COURSE
WHERE credit_hrs > 3;
```

Output

CNAME	CREDIT_HRS
DBMS	4

Result

The query was executed successfully and displayed all courses having credit hours greater than 3. From the available records in the COURSE table, the course **DBMS** satisfies the specified condition with **4 credit hours**. This confirms that the filtering condition in the **WHERE** clause has been applied correctly and only the required course has been retrieved from the database.

Question 6

Retrieve the names of students who received grade 'A'.

Explanation

The STUDENT table contains information about students such as their names, student numbers, class, and major. The GRADE_REPORT table stores the grades obtained by students in different course sections. To identify students who have secured a particular grade, information from both tables must be combined using the common attribute **snumber**.

The query joins the STUDENT and GRADE_REPORT tables by matching the student numbers present in both tables. The **WHERE** clause is used to establish the relationship between the tables and to filter the records based on the specified grade value. In this case, only those records where the grade is equal to 'A' are selected. The **SELECT** statement retrieves the names of the students who satisfy this condition.

This type of query is useful for identifying high-performing students, preparing merit lists, generating academic reports, and analyzing student performance. By combining data from multiple tables and applying filtering conditions, the database can efficiently provide meaningful information to users.

When the query is executed, the database searches for matching student records and returns only those students who have obtained the grade 'A' in their respective course sections.

SQL Query

```
SELECT S.name
FROM STUDENT S, GRADE_REPORT G
WHERE S.snumber = G.snumber
AND G.grade = 'A';
```

Output

NAME
Rahul
Anjali

Result

The query was executed successfully and displayed the names of students who obtained grade 'A' in their course sections. The result shows that **Rahul** and **Anjali** satisfied the specified condition and were therefore included in the output. This demonstrates the use of table joining and filtering techniques to retrieve specific academic information from the database. The output confirms that the query correctly identified all students who achieved the highest grade and displayed their names successfully.

Question 7

Display the number of students in each class.

Explanation

The STUDENT table stores information about students, including their name, student number, class, and major. Educational institutions often require summarized information about the distribution of students across different classes for academic planning and administrative purposes.

The objective of this query is to determine the total number of students belonging to each class category. To achieve this, the query uses the **COUNT()** aggregate function, which counts the number of records present in each group. The **GROUP BY** clause is used to group all students according to their class value, such as junior or senior.

When the query is executed, the database scans all records in the STUDENT table and organizes them into separate groups based on the class attribute. The COUNT() function then calculates the number of students present in each group. The resulting output displays the class name along with the corresponding number of students.

Such aggregate queries are widely used in database systems to generate statistical summaries, prepare reports, and analyze data efficiently. Instead of displaying individual student records, the query provides a concise overview of student distribution among different classes.

SQL Query

```
SELECT class,  
       COUNT(*) AS total_students  
FROM STUDENT  
GROUP BY class;
```

Output

CLASS	TOTAL_STUDENTS
senior	2
junior	1

Result

The query was executed successfully and displayed the total number of students belonging to each class category. The output shows that there are **2 students** in the **senior** class and **1 student** in the **junior** class. This confirms that the COUNT() aggregate function and GROUP BY clause were applied correctly to summarize the student data. The generated result provides a clear overview of the distribution of students across different classes and demonstrates the use of aggregation operations in SQL for producing meaningful statistical information.